DEVOPS CAPSTONE

Issue Tracker, end to end

From a laptop to a monitored Kubernetes deployment: tests, image scanning, a registry, Helm, Ingress, autoscaling, Prometheus and GitOps.

Rajveer Bishnoi · Enrollment 24BCS10404 · Roll 10404

What the project is

A small but complete issue tracker: create an issue, move it through open, in progress and done, filter it, and see counts. The application is deliberately modest so that the delivery pipeline around it can be the real subject.

3
services: React, FastAPI, PostgreSQL

5
CI jobs, all green

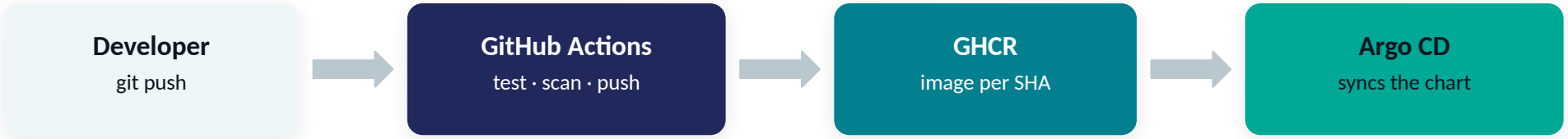
8
backend tests, all passing

0
fixable HIGH or CRITICAL CVEs

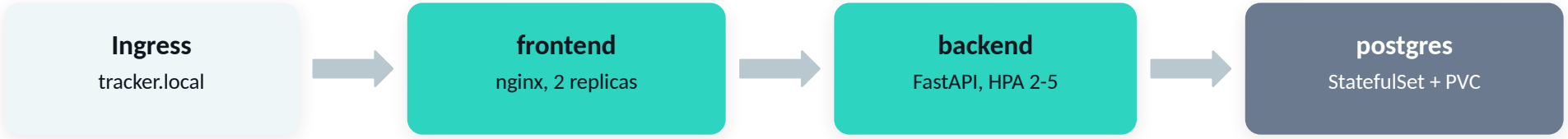
The numbers above come from pipeline run 38892471244 on this repository: the two images are built, scanned and pushed to GHCR, then deployed to a kind cluster and smoke tested before the run is allowed to pass.

Architecture

Delivery path



Runtime, inside the cluster



Prometheus discovers backend Pods by annotation, so every replica the autoscaler adds is scraped without configuration. Grafana reads Prometheus. The Ingress is the only way in: postgres has no Service exposed outside the namespace.

Tech stack

1	**Application**	React 18 with Vite, FastAPI, SQLAlchemy 2, Alembic migrations, PostgreSQL 16
2	**Containers**	Multi-stage Dockerfiles, non-root users, nginx-unprivileged, Docker Compose
3	**Pipeline**	GitHub Actions, GHCR, Trivy, Helm, kind in the runner
4	**Kubernetes**	Minikube, ingress-nginx, metrics-server, Helm chart with dev and prod values
5	**Operations**	Prometheus with alert rules, Grafana dashboard, Argo CD, Terraform for the network

The application and its API

Nine endpoints, three of them operational

```
GET    /health           liveness, no dependencies
GET    /ready            readiness, checks the database
GET    /metrics          Prometheus exposition

GET    /api/issues       list, filterable by status
GET    /api/issues/stats counts by status and priority
POST   /api/issues       create, validates priority
PUT    /api/issues/{id}  update
DELETE /api/issues/{id}  delete
```

```
collected 8 items
8 passed, 1 warning in 0.11s
```

Why the split matters

A liveness probe that touched the database would restart a healthy Pod whenever the database blinked. /health answers from the process alone; /ready is the one that checks the database, so a Pod is pulled out of the Service instead of killed.

Tested, not asserted

Eight pytest cases cover the create/read/update/delete cycle, validation rejection, stats, readiness and the metrics counter.

The API is small on purpose. What matters is that every endpoint the cluster depends on, including the two probes and the metrics route, is covered by a test that runs before any image is built.

Containers and Compose

1

Multi-stage

The frontend is built with Node and shipped on nginx; the toolchain never reaches the runtime image.

2

Non-root

Both images run as a fixed non-root UID, with capabilities dropped in the Kubernetes manifests.

3

Slim bases

Debian slim and Alpine, upgraded at build time so base-image CVEs do not ride along.

4

One command

docker compose up -d --build brings up the database, the API and the UI together.

```
$ docker compose up -d --build
  tracker-db        healthy
  tracker-backend   started   (alembic upgrade head on boot)
  tracker-frontend  started   (nginx proxies /api to the backend)
```

The pipeline

Three stages. Nothing reaches the registry until the tests and the scanner have both agreed.

1

backend tests

pytest, JUnit artifact

1

frontend build

Vite, dist artifact

2

build, scan, push

buildx, Trivy, GHCR

3

deploy to kind

helm upgrade, smoke test

What the smoke test actually proves

```
NAME: tracker          STATUS: deployed       REVISION: 1
{"status":"ready"}
{"title":"created by CI","status":"open","priority":"high","id":1}
{"total":1,"by_status":{"open":1,"in_progress":0,"done":0}}
tracker_issues_created_total 0.0
```

The security gate

Policy

```
severity:        HIGH,CRITICAL
ignore-unfixed:  true     # no patch anywhere yet -> report,
do not block
exit-code:       1        # anything else fails the job
before the push
```

The reasoning: a vulnerability we can clear by bumping a version should stop the build, because the fix is cheap. One with no fix available anywhere should be visible but must not freeze every release.

Result on this repository

0

fixable HIGH or CRITICAL findings on either image, scanned in the run that pushed them.

```
issue-tracker-backend  (debian 13.7)   0
issue-tracker-frontend (alpine 3.23.4) 0
bandit  No issues identified.  High: 0
semgrep Findings: 0 (0 blocking)
```

A scan that fails the build is only worth having if the fix is cheap. Base-image updates are, which is what makes a hard failure on fixable findings a reasonable default rather than an obstacle.

Kubernetes and Helm

One chart creates the whole release; three value files point it at different environments.

postgres	StatefulSet with a volumeClaimTemplate, headless Service, pg_isready probes, credentials from a Secret
backend	Two replicas, startup / readiness / liveness probes, runAsNonRoot, capabilities dropped, scrape annotations
frontend	nginx serving the built bundle, config from a ConfigMap that proxies /api to the backend Service
Ingress + HPA	tracker.local routes / and /api; the autoscaler holds the backend between 2 and 5 replicas at 60% CPU

A detail that matters

The backend Deployment carries a checksum of the ConfigMap and Secret as an annotation. Change the configuration and the checksum changes, so the Pods roll. Without it, a config change updates the objects and nothing restarts to pick it up.

```
helm lint   ok
helm upgrade --install tracker
  --wait --timeout 5m
STATUS: deployed
```

Observability and autoscaling

1

Discovery by annotation

Prometheus keeps only Pods annotated prometheus.io/scrape. The chart puts that annotation on the backend, so a Pod the HPA creates is scraped within one interval, with no edit to the scrape config.

2

What the app exposes

A request counter by route and status, a latency histogram, an in-flight gauge, and a counter of issues created. The last one is a business signal, not an infrastructure one.

3

Alerts, not just graphs

BackendHighErrorRate fires when 5xx stays above five percent for two minutes. BackendDown fires when the target disappears. A dashboard nobody is watching catches nothing.

The autoscaler and the scrape configuration are designed together: neither one needs to know how many replicas there will be.

GitOps with Argo CD

An Application object points at the chart in this repository with automated sync, prune and self-heal.

Git becomes the deployment

Change a value, commit, push. Argo CD notices the new commit and reconciles the cluster. No helm or kubectl command is run by a person.

The lesson from the run

ignoreDifferences on /spec/replicas has to name the HPA-managed Deployment. Applied to every Deployment, it made Argo CD ignore a deliberate frontend change too.

```
syncPolicy:
  automated: { prune: true, selfHeal: true }
ignoreDifferences:
  - group: apps
    kind: Deployment
    name: tracker-backend      # scoped, not blanket
    jsonPointers: [/spec/replicas]
```

GitOps and autoscaling disagree about who owns the replica count. Say who owns it, and say it for the right object.

The troubleshooting drill

Two faults introduced into the running release, each diagnosed from the cluster before any fix was applied.

1

Wrong database host

Symptom: Rollout stuck, new Pod 0/1, old Pods still serving traffic

Found with: kubectl logs showed name resolution failing; describe showed the readiness probe never passing

helm rollback

2

Ingress to a Service that does not exist

Symptom: UI loads, every API call returns 503

Found with: describe ingress named the missing Service; the backend answered fine from inside the cluster

re-render the Ingress

What the project shows

1. A readiness probe that checks the database turned a bad config change into a stuck rollout instead of an outage. The old Pods never stopped serving.

2. A scan that fails the build is useful only when the fix is cheap, so block on fixable findings and merely report the rest.

3. GitOps and autoscaling both claim the replica count; the conflict has to be declared, and declared narrowly.

4. Running the whole stack on a local cluster first made the cloud step a change of target rather than a different project.

github.com/RAJVEER42/DevOps